CS 432 Distributed Systems	Lab (3)

Section Content:

- **Introduction to Concurrency**
- **Processes and Threads**

Introduction to Concurrency

What is a task?

A **task** is one piece of work that we want the computer to do.

- Download a file.
- Play a song.
- Send an email.

Example: Your computer is running NetBeans, your browser is downloading a file, and Task Manager is open. Each of these is a task.

What is sequential execution?

Sequential execution means the computer does the tasks **one after another**. The next task starts only when the one before it is finished.

Example: A server receives two requests. Request A takes 4 seconds and Request B takes 3 seconds. The server finishes A, then starts B.

- Total time = 4 + 3 = 7 seconds.
- The user who sent B waits 4 seconds before the server even starts on their request.

What is concurrent execution?

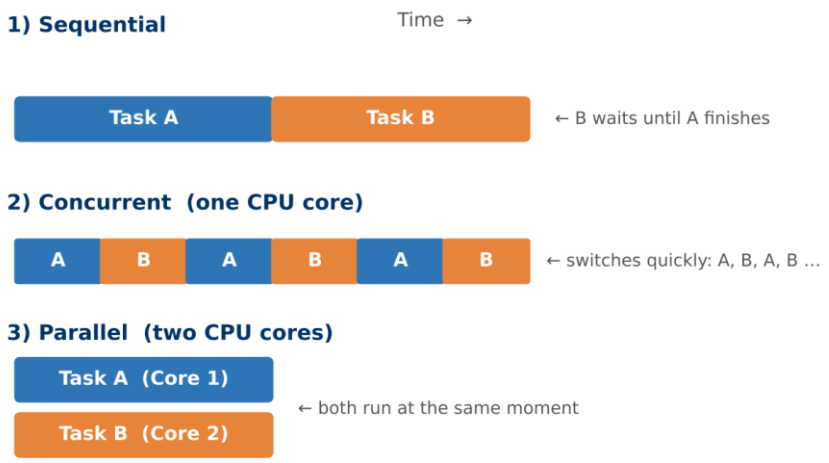
Concurrent execution means that **several tasks are in progress during the same period of time**. The computer does not wait for one task to finish. It moves between the tasks.

Example: The same server now handles A and B concurrently. While A is waiting for the database, the server works on B. Neither user waits for the other to finish.

> **Easy way to remember**
> Sequential = **finish A**, then start B.
> Concurrent = A and B are **both in progress** in the same period.

Difference between concurrency and parallelism

These two words sound similar, but they are **not the same**.



	Concurrency	Parallelism
Main idea	Many tasks are **in progress** at the same time	Many tasks **run at exactly** the same moment
Workers needed	Can work with **one** worker (one CPU core)	Needs **two or more** workers (CPU cores)
How it works	The worker **switches** quickly between tasks	Each worker has its **own** task

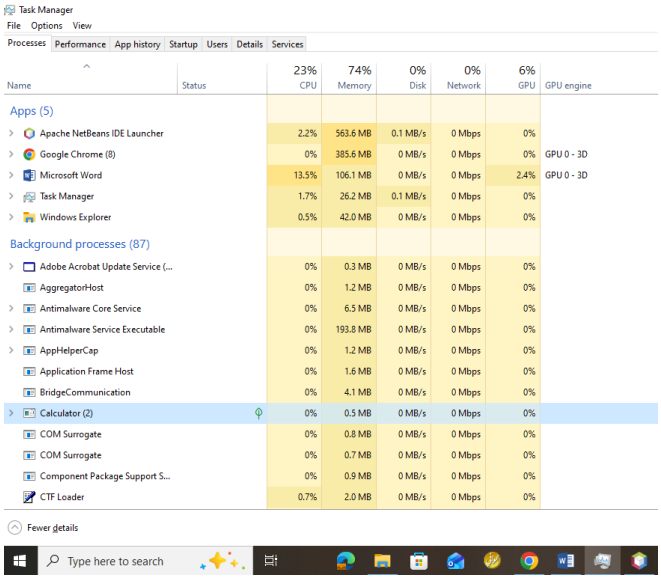
> **Key sentence**
> **Concurrency** is about **dealing with** many things at once.
> **Parallelism** is about **doing** many things at once.

Processes and Threads

What is a process?

A **process** is a **program that is running**. When you double-click an app (Chrome, Word, a Java program), the operating system creates a process for it.

- Every process gets **its own memory** (its own private space).
- You can see your processes in the Task Manager.

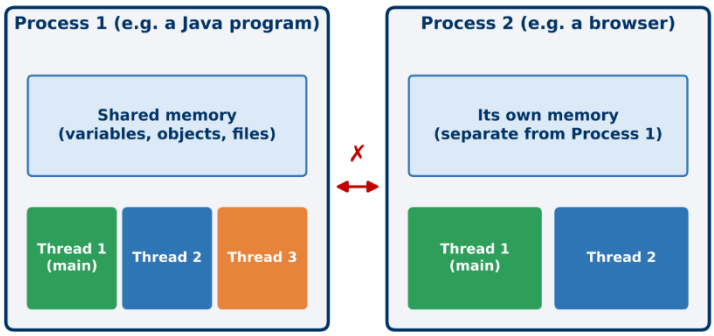


What is a thread?

A **thread** is a **small worker inside a process**. It is the smallest unit of work that the CPU can run.

- A process has **at least one** thread. It can have many.
- All threads of the same process **share the same memory**.

> **Think of a restaurant**
>
> The **process** = the whole kitchen. The **threads** = the cooks. All cooks work in the **same kitchen** and use the **same fridge** (shared memory).



Difference between a process and a thread

	Process	Thread
Meaning	A running program	A small worker inside a process
Memory	Has its **own** memory	**Shares** memory with other threads
Creating it	Heavy (takes more time and resources)	Light (faster and cheaper)
Talking to each other	Difficult (needs special tools)	Easy (they use the same variables)
If one crashes	The other processes usually **keep running**	The **whole process** can be affected

Main thread and worker threads

- When a Java program starts, Java creates **one thread for us automatically**. It is called the **main thread**.
- The main thread runs the `main()` method.
- The main thread can create **more threads**. These extra threads are called **worker threads**. Each worker does a job for us.

Programs

Concurrent Execution in a Distributed System This program demonstrates the Concurrent Execution characteristic of a distributed system. Two independent nodes, Node-A and Node-B, perform their tasks simultaneously using separate threads. The main program waits for both threads to complete before displaying a final message.

Code

```
package concurrentexecutiondemo;
public class ConcurrentExecutionDemo {
    public static void main(String[] args) {
      // Node-A task
      Thread nodeA = new Thread(() -> {
        for (int i = 1; i <= 3; i++) {
        System.out.println("Node-A processing task " + i);
      }
      });
      // Node-B task
      Thread nodeB = new Thread(() -> {
       for (int i = 1; i <= 3; i++) {
           System.out.println("Node-B processing task " + i);
        }
       });
      // Start both nodes concurrently
        nodeA.start();
        nodeB.start();
      // Wait for both nodes to finish
      try {
        nodeA.join();
        nodeB.join();
      }
      catch (InterruptedException e) {
         e.printStackTrace();
      }
      // Final message
      System.out.println("All nodes have completed their work.");
    }
}
```

Output

```
Node-B processing task 1
Node-B processing task 2
Node-A processing task 1
Node-B processing task 3
Node-A processing task 2
Node-A processing task 3
All nodes have completed their work.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Multiple clients on one machine and have them send requests simultaneously to a server in Java, you can create multiple client threads. Each thread acts as an independent client with its own socket connection.

Server Code

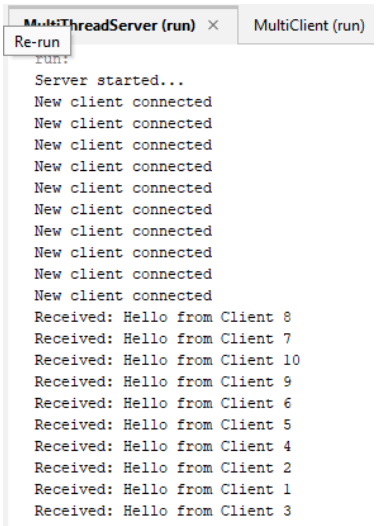
```
package multithreadserver;
import java.io.*;
import java.net.*;
public class MultiThreadServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server started...");
            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("New client connected");
                ClientHandler handler = new ClientHandler(clientSocket);
                new Thread(handler).start();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
class ClientHandler implements Runnable {
    private Socket socket;
    public ClientHandler(Socket socket) {
        this.socket = socket;
    }
    @Override
    public void run() {
        try {
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            String message = in.readLine();
            System.out.println("Received: " + message);
            out.println("Server received: " + message);
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
```

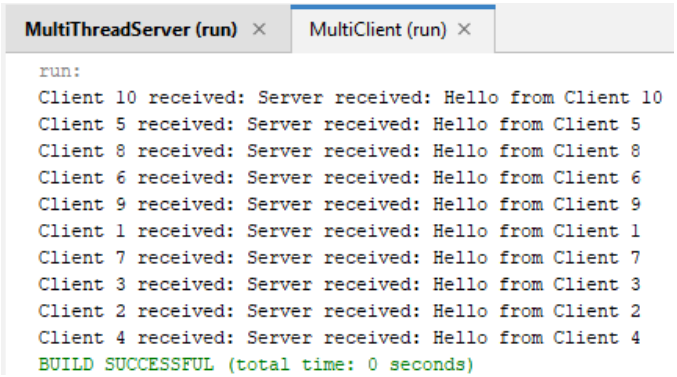
```
        }
    }
}
```

Clients Code

```
package multiclient;
import java.io.*;
import java.net.*;
public class MultiClient {
    public static void main(String[] args) {
        int numberOfClients = 10;
        for (int i = 1; i <= numberOfClients; i++) {
            final int clientId = i;
            Thread clientThread = new Thread(() -> {
              try {
                Socket socket = new Socket("localhost", 5000);
                PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
                BufferedReader in = new BufferedReader( new InputStreamReader
(socket.getInputStream()));
                String msg = "Hello from Client " + clientId;
                out.println(msg); String response = in.readLine();
                System.out.println( "Client " + clientId + " received: " + response);
                socket.close();
              } catch (Exception e) {
                e.printStackTrace();
              }
            });
            clientThread.start();
        }
    }
}
```

Output





Concurrent Execution in a Distributed System This program demonstrates Concurrent Execution, one of the fundamental characteristics of distributed systems. It simulates three independent nodes (Node-A, Node-B, and Node-C) executing tasks simultaneously using separate threads. Since the threads run concurrently, the order of execution is determined by the operating system's scheduler and may vary each time the program runs.

Code

```
package concurrentexecutiondemo3;
public class ConcurrentExecutionDemo3 {
    public static void main(String[] args) {
       // Node-A task
      Thread nodeA = new Thread(() -> {
       for (int i = 1; i <= 3; i++) {
       System.out.println("Node-A processing task " + i);
      }
      });
      // Node-B task
      Thread nodeB = new Thread(() -> {
       for (int i = 1; i <= 3; i++) {
           System.out.println("Node-B processing task " + i);
        }
       });
      // Node-C task
      Thread nodeC = new Thread(() -> {
       for (int i = 1; i <= 3; i++) {
           System.out.println("Node-C processing task " + i);
        }
       });
      // Start both nodes concurrently
        nodeA.start();
        nodeB.start();
        nodeC.start();
      // Wait for both nodes to finish
      try {
        nodeA.join();
        nodeB.join();
        nodeC.join();
      }
      catch (InterruptedException e) {
         e.printStackTrace();
      }
      // Final message
```

```
            System.out.println("All nodes have completed their work.");
        }

    }
```

Output

```
Node-C processing task 1
Node-C processing task 2
Node-A processing task 1
Node-B processing task 1
Node-C processing task 3
Node-B processing task 2
Node-B processing task 3
Node-A processing task 2
Node-A processing task 3
All nodes have completed their work.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Assignment

Simple Program: Concurrent Execution This program demonstrates Scalability and Concurrent Execution in a distributed system. It simulates a server receiving multiple user requests at the same time. Instead of processing requests sequentially, the server creates a separate thread for each request, allowing many requests to be handled concurrently.